Robot Dynamics, Control and Motion Planning

Lecture 5A

Jitendra Malik

Some slides sourced from Lynch and Park, Modern Robotics (credited on slide)

Three topics

- Robot Dynamics
- Robot Control
- Robot Motion Planning

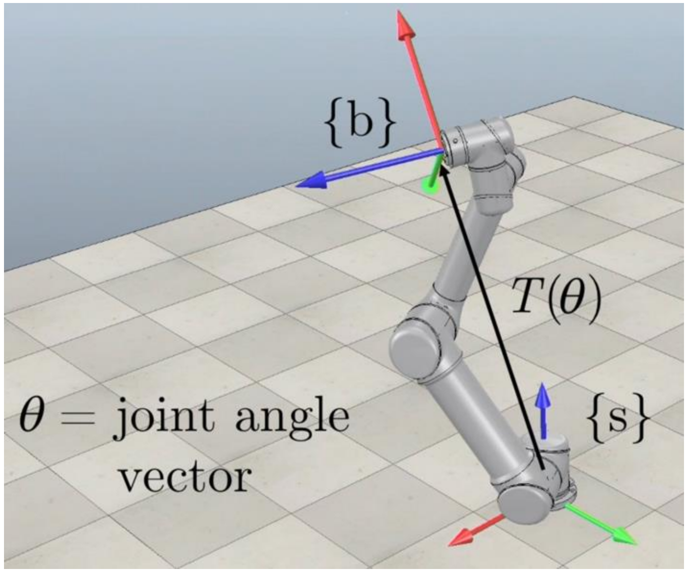
Important concepts, symbols, and equations

Forward kinematics of a serial chain:

Given

- $M = T_{sb}(0) \in SE(3)$, the configuration of the end-effector frame $\{b\}$ at the home configuration $\theta = 0$,
- the screw axes for each joint at $\theta = 0$, and
- the joint vector θ,

find $T_{sb}(\theta) \in SE(3)$.



Important concepts, symbols, and equations (cont.)

- For screw axes S expressed in $\{s\}$, the **product of exponentials** (PoE) in the space frame is

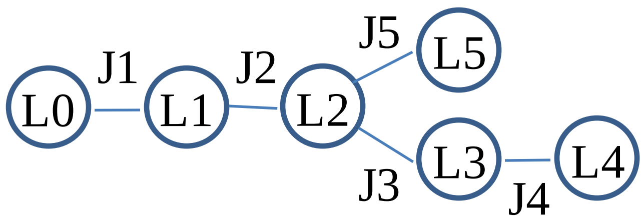
$$T(\theta) = e^{[\mathcal{S}_1]\theta_1} \cdots e^{[\mathcal{S}_n]\theta_n} M$$

- For screw axes B expressed in $\{b\}$, the PoE is

$$T(\theta) = M e^{[\mathcal{B}_1]\theta_1} \cdots e^{[\mathcal{B}_n]\theta_n}$$

Important concepts, symbols, and equations (cont.)

The **Universal Robot Description Format (URDF)** is an XML file describing the kinematics, inertial properties, and geometry of a tree-structured robot.



the **joint** element (kinematics)

```
<joint name=“joint2” type=“revolute”>
  <parent link=“link1”/>
  <child link=“link2”/>
  <origin rpy=“0 1.5708 0” xyz=“0 0.5 0”>
  <axis xyz=“0 1 0”>
  <limit lower=“-3.0” upper=“3.0”>
</joint>
```

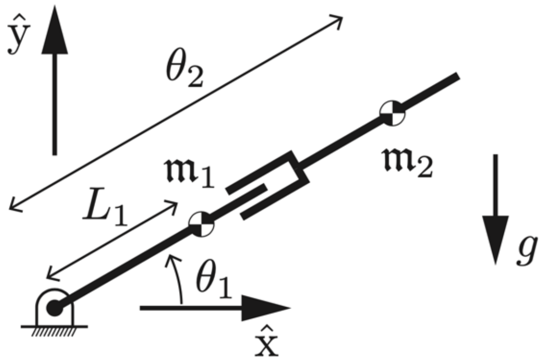
the **link** element (inertial properties)

```
<link name=“link2”>
  <inertial>
    <origin rpy=“0 0 0” xyz=”0 0 0.5”>
    <mass value=“2.25”>
    <inertia ixx=“1.0” ixy=“0” ixz=“0”
             iyy=“2.0” iyz=“0” izz=“3.0”/>
  </inertial>
</link>
```

Important concepts, symbols, and equations

- **forward dynamics** (for simulation): $\theta, \dot{\theta}, \tau \rightarrow \ddot{\theta}$
- **inverse dynamics** (for control): $\theta, \dot{\theta}, \ddot{\theta} \rightarrow \tau$
- two equivalent approaches to computing the dynamics:
  - Lagrangian (variational, based on energy)
  - Newton-Euler ("f = ma" for the rigid bodies)

RP in gravity



$$\mathcal{P}_1 = \mathfrak{m}_1 g y_1 = \mathfrak{m}_1 g L_1 \sin\theta_1$$
$$\mathcal{P}_2 = \mathfrak{m}_2 g y_2 = \mathfrak{m}_2 g \theta_2 \sin\theta_1$$

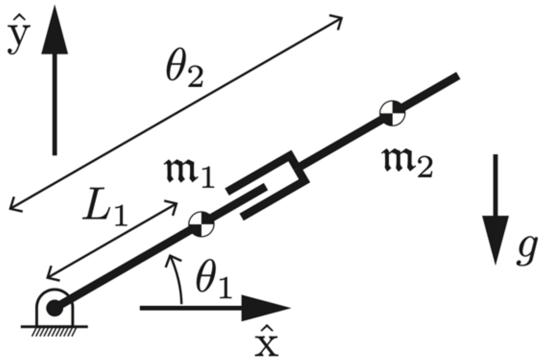
$$\mathcal{K}_1 = \frac{1}{2}\mathfrak{m}_1(\dot{x}_1^2 + \dot{y}_1^2) + \frac{1}{2}\mathcal{I}_1\dot{\theta}_1^2 = \frac{1}{2}(\mathcal{I}_1 + \mathfrak{m}_1 L_1^2)\dot{\theta}_1^2$$
$$\mathcal{K}_2 = \frac{1}{2}\mathfrak{m}_2(\dot{x}_2^2 + \dot{y}_2^2) + \frac{1}{2}\mathcal{I}_2\dot{\theta}_1^2 = \frac{1}{2}\left((\mathcal{I}_2 + \mathfrak{m}_2\theta_2^2)\dot{\theta}_1^2 + \mathfrak{m}_2\dot{\theta}_2^2\right)$$

$$\mathcal{L} = \mathcal{K}_1 + \mathcal{K}_2 - \mathcal{P}_1 - \mathcal{P}_2 = \frac{1}{2}\mathfrak{m}_2\theta_2^2\dot{\theta}_1^2 + \ldots$$

$$= L_1 + \ldots$$

https://www.feynmanlectures.caltech.edu/II_19.html

RP in gravity



$$\tau = M(\theta)\ddot{\theta} + c(\theta, \dot{\theta}) + g(\theta)$$

$$M(\theta) = \begin{bmatrix} \mathcal{I}_1 + \mathcal{I}_2 + \mathfrak{m}_1 L_1^2 + \mathfrak{m}_2 \theta_2^2 & 0 \\ 0 & \mathfrak{m}_2 \end{bmatrix}$$

$$c(\theta, \dot{\theta}) = \begin{bmatrix} 2\mathfrak{m}_2 \theta_2 \dot{\theta}_1 \dot{\theta}_2 \\ -\mathfrak{m}_2 \theta_2 \dot{\theta}_1^2 \end{bmatrix}$$

$$g(\theta) = \begin{bmatrix} (\mathfrak{m}_1 L_1 + \mathfrak{m}_2 \theta_2) g \cos\theta_1 \\ \mathfrak{m}_2 g \sin\theta_1 \end{bmatrix}$$

Important concepts, symbols, and equations (cont.)

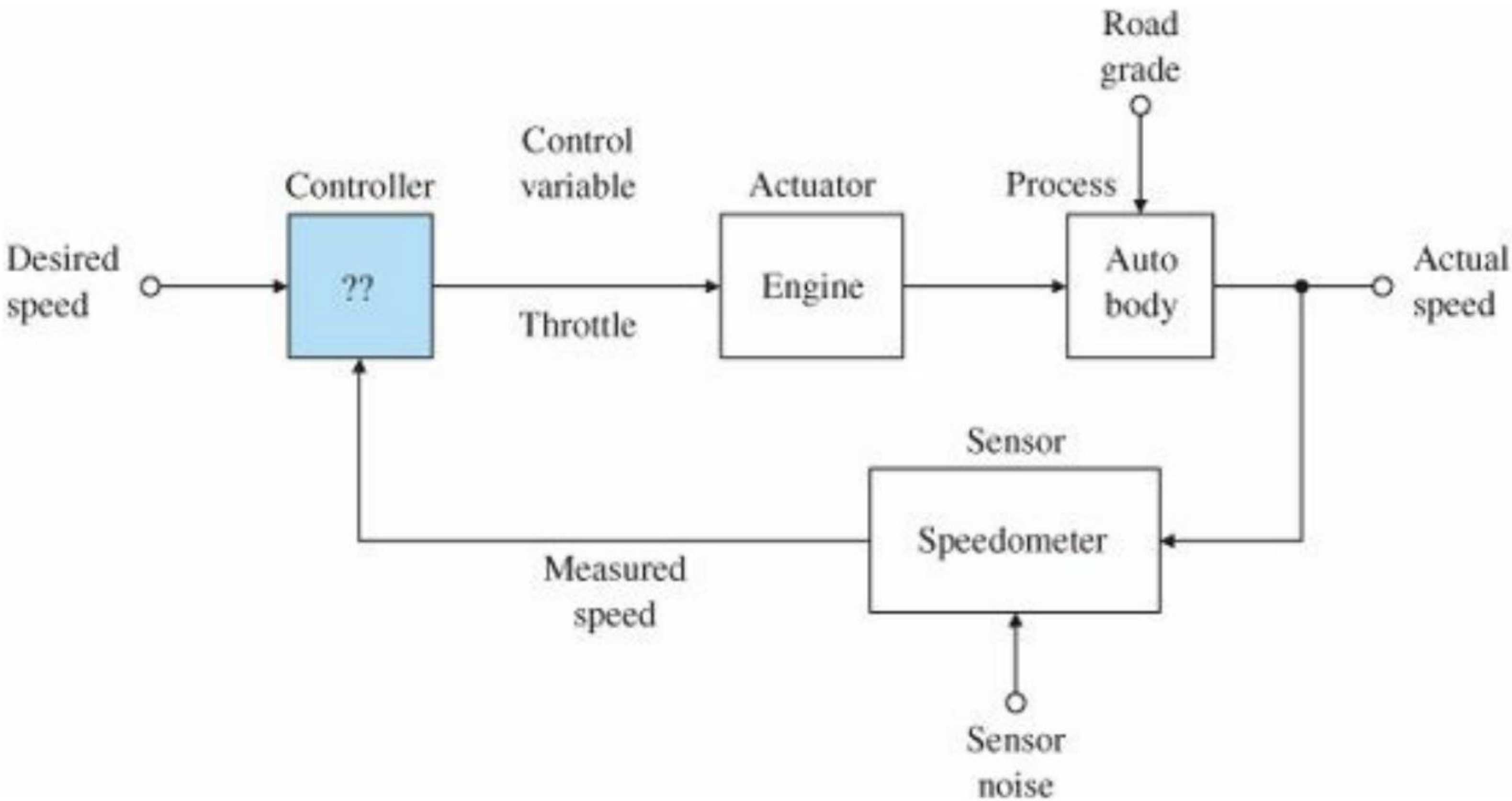
Standard forms of dynamic equations:

$$
\begin{aligned}
\tau &= M(\theta)\ddot{\theta} + h(\theta,\dot{\theta}) \\
&= M(\theta)\ddot{\theta} + c(\theta,\dot{\theta}) + g(\theta) \\
&= M(\theta)\ddot{\theta} + \dot{\theta}^{\mathrm{T}}\Gamma(\theta)\dot{\theta} + g(\theta) \\
&= M(\theta)\ddot{\theta} + C(\theta,\dot{\theta})\dot{\theta} + g(\theta)
\end{aligned}
\qquad + J^{\mathrm{T}}(\theta)\mathcal{F}_{\mathrm{tip}}
$$

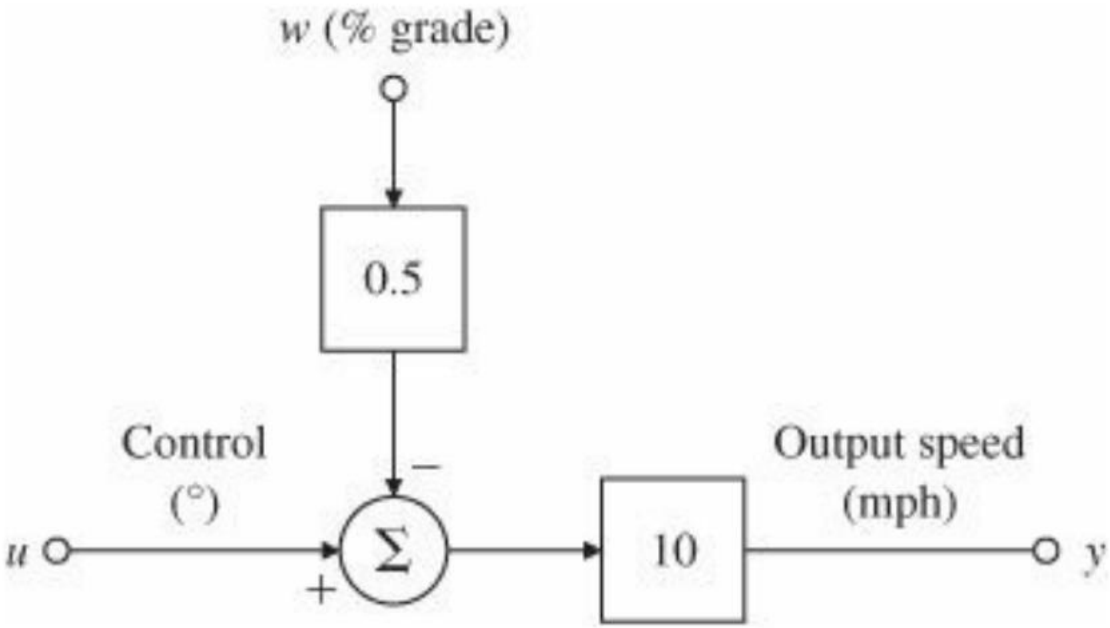
$M(\theta)$	$n \times n$ symmetric positive definite **mass matrix**
$g(\theta)$	gravity (potential) terms
$c(\theta,\dot{\theta})$	velocity-product terms
$\Gamma(\theta)$	$n \times n \times n$ tensor of **Christoffel symbols** (due to nonzero $\partial M/\partial\theta$)
$C(\theta,\dot{\theta})$	**Coriolis matrix**

Cruise controller

Source: Feedback control of dynamic systems, Franklin et al



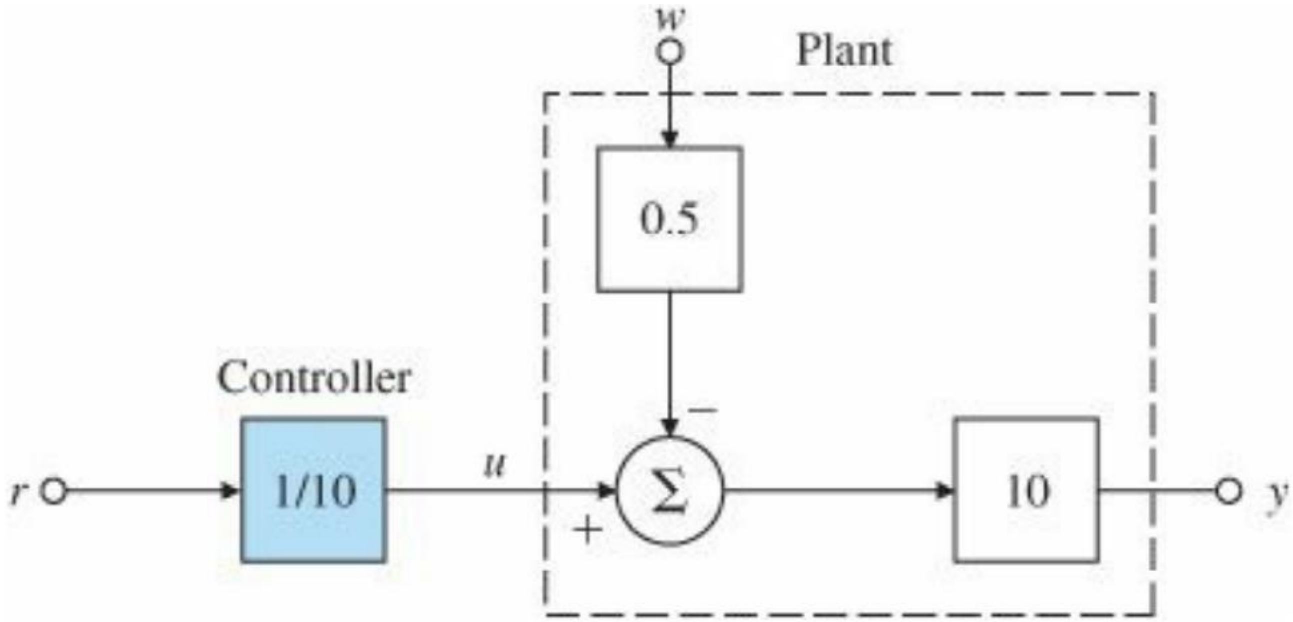
Model



1 deg change in throttle angle causes a speed change by 10 mph
1% change in grade causes a speed change by 5 mph

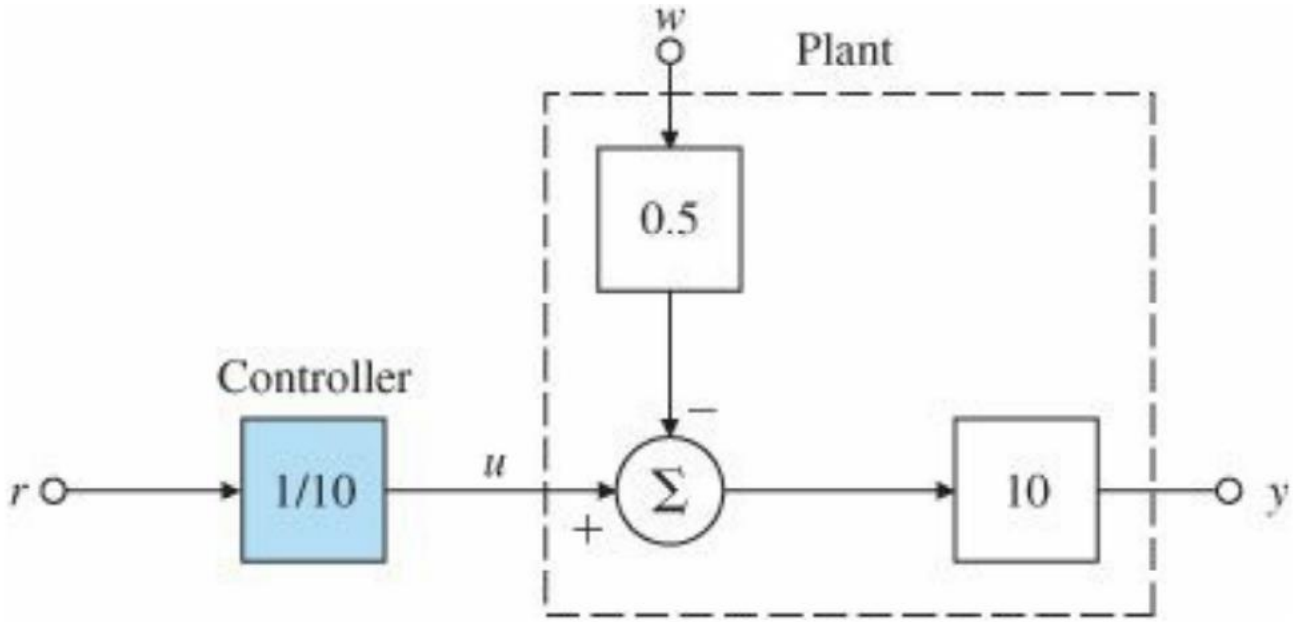
Open loop controller

(also called Feedforward controller)



Open loop controller

(also called Feedforward controller)

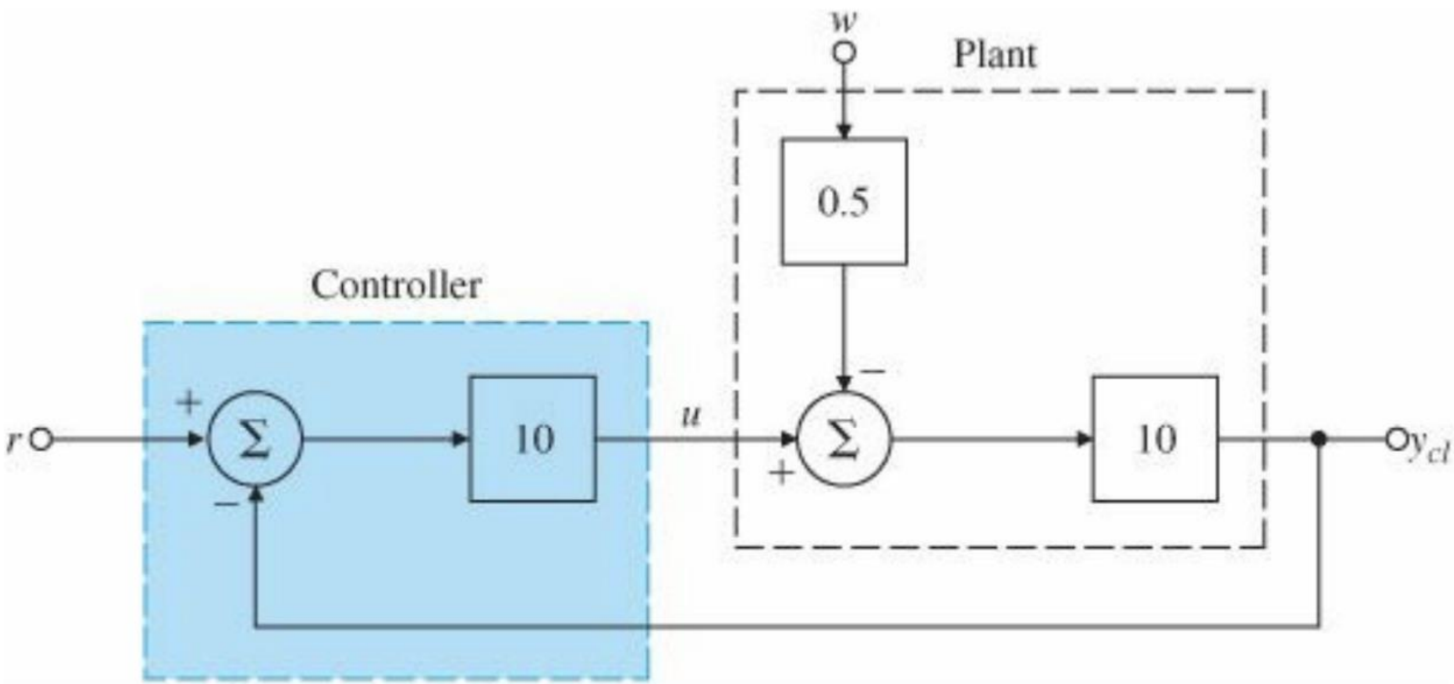


$y = 10\,(u - 0.5\,w)$
$y = 10\,(0.1\,r - 0.5\,w)$
$y = r - 5\,w$

If the car is going up a 1% grade slope, its speed will be 5 mph less than reference

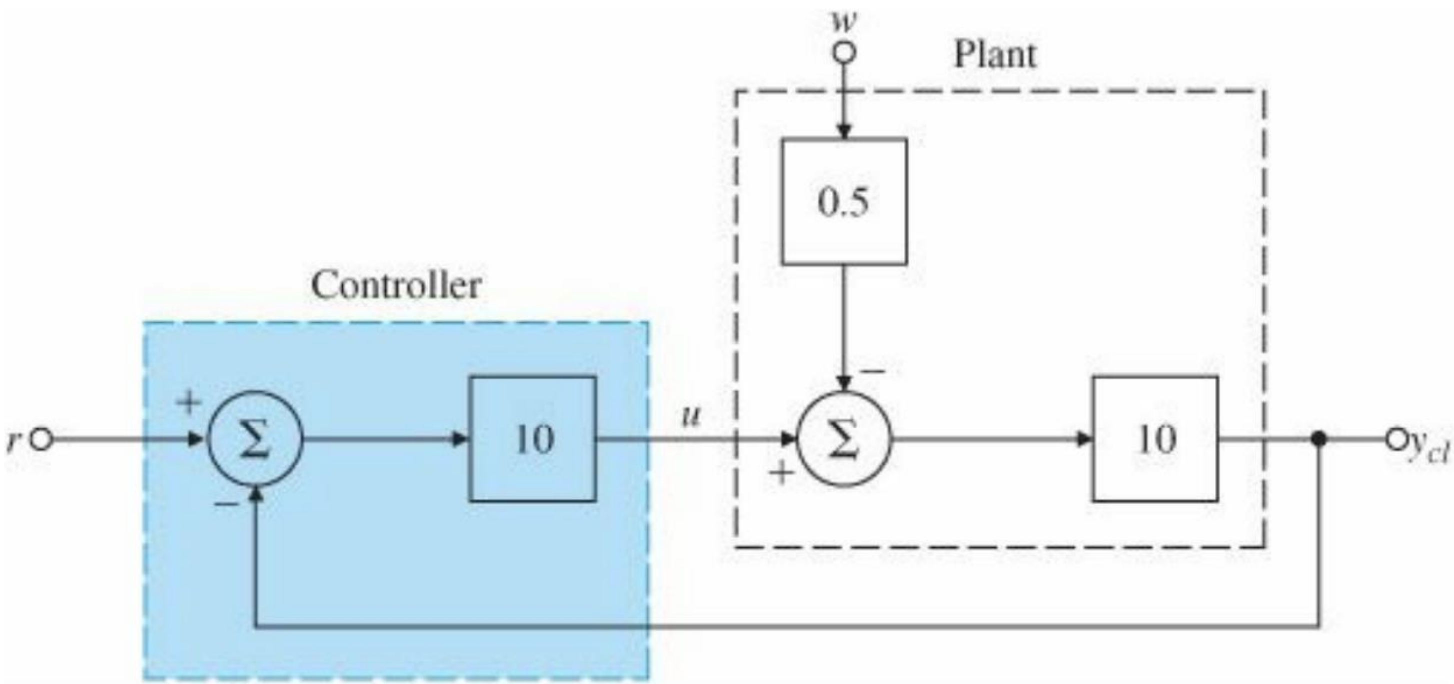
Closed Loop Controller

Also called Feedback controller



Closed Loop Controller

Also called Feedback controller

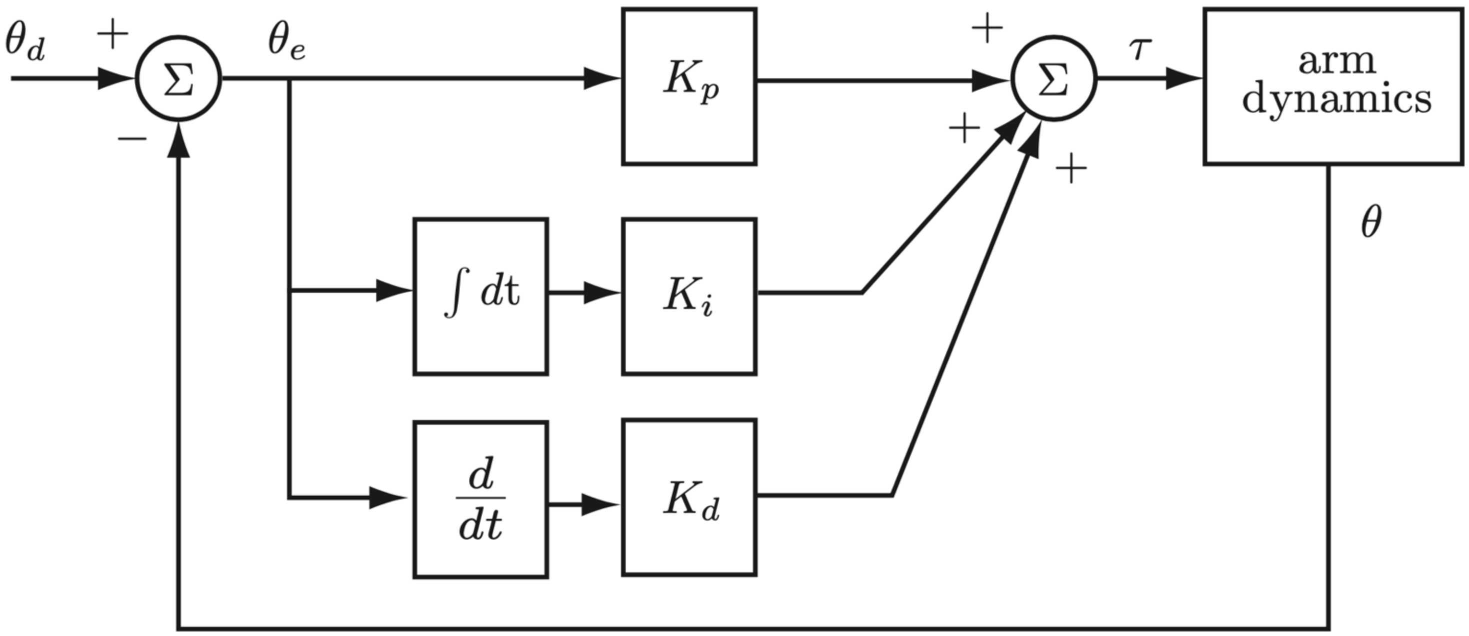


$u = 10\ (r - y)$
$y = 10\ (u - 0.5\ w)$
$y = 10\ (10r - 10y - 0.5w)$

$y = 0.99\ r - 0.05\ w$

The effect of the disturbance is reduced by 100x

PID controller



Important concepts, symbols, and equations (cont.)

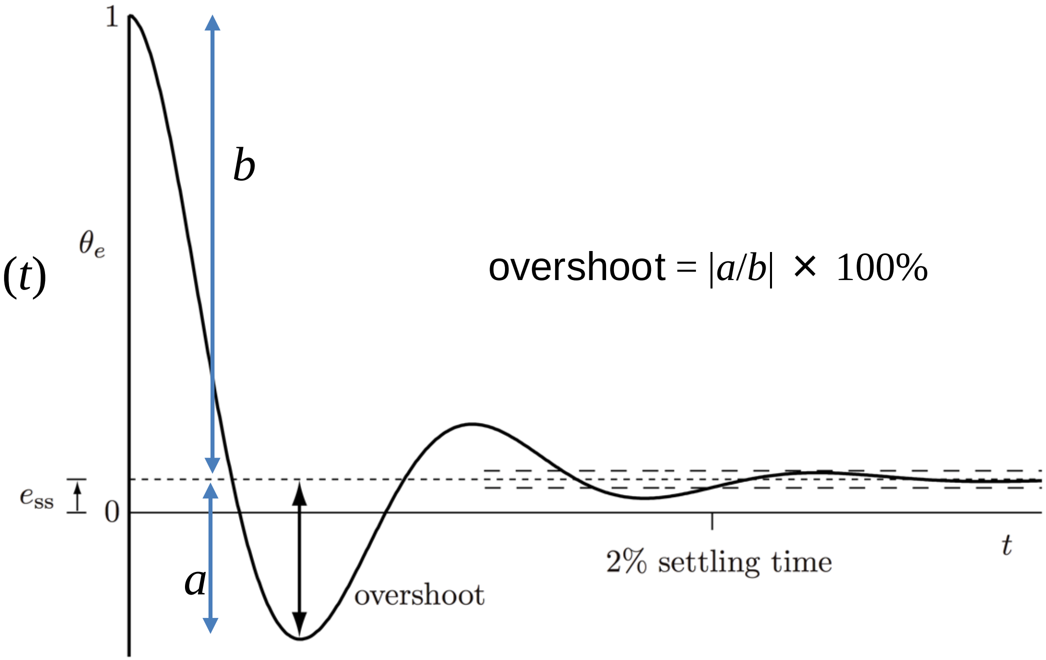
For motion control,

reference: $\theta_d(t)$
actual: $\theta(t)$
error: $\theta_e(t) = \theta_d(t) - \theta(t)$

Unit step error response:
$\theta_e(t)$ starting from $\theta_e(0) = 1$

Steady-state error response: e_{ss}

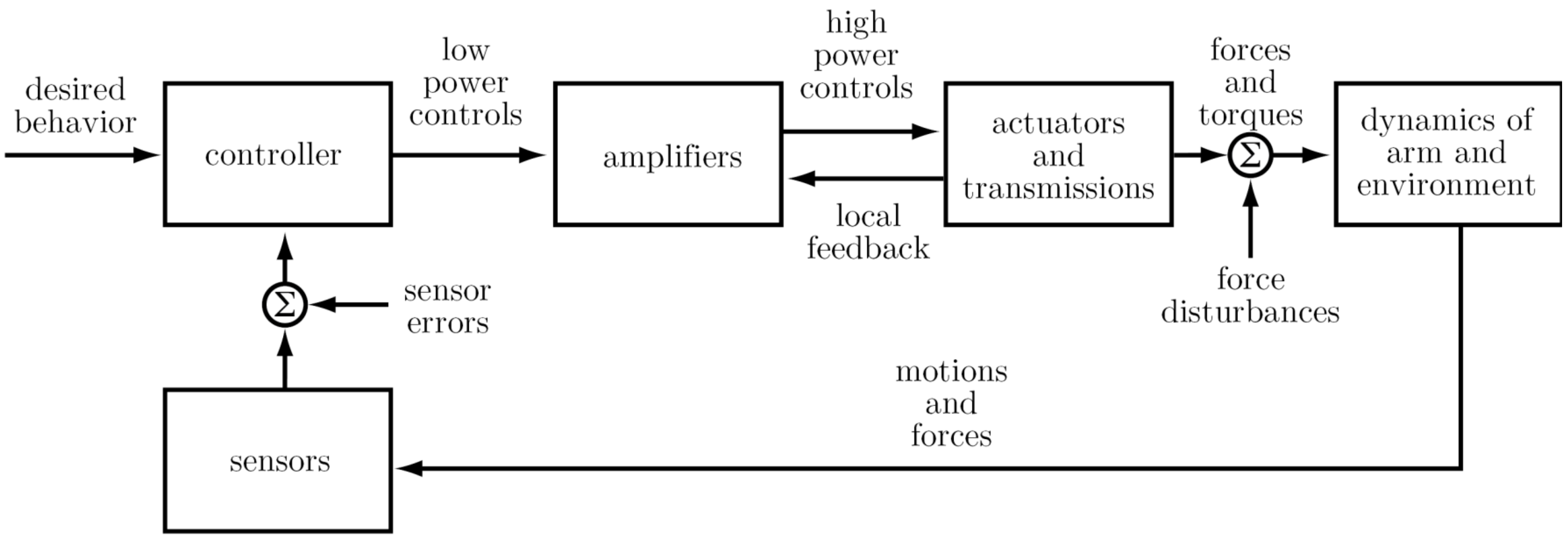
Transient error response:
overshoot, settling time



overshoot = $|a/b| \times 100\%$

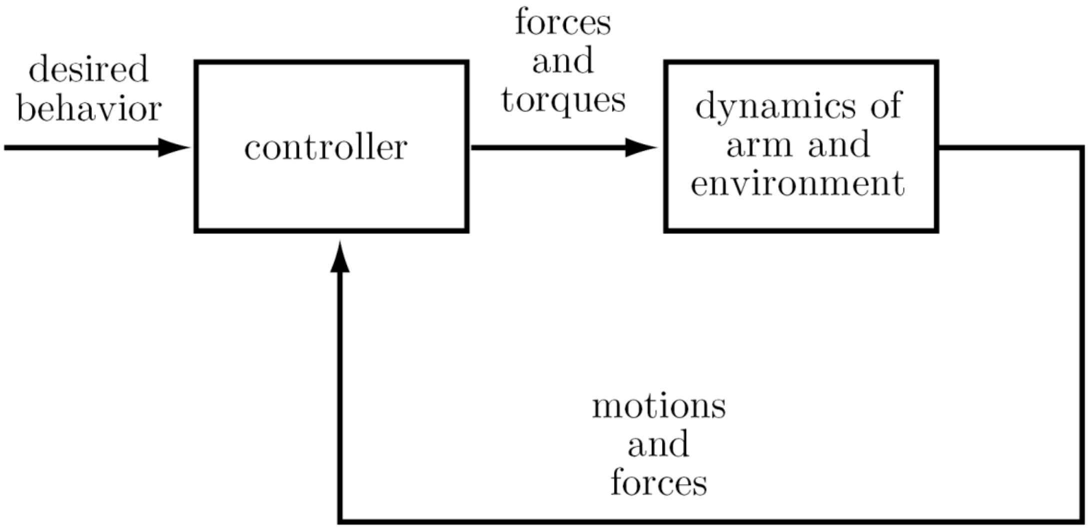
Back to robots…

Control system block diagram:



Important concepts, symbols, and equations (cont.)

Simplified block diagram:



Also assuming continuous-time (not discrete-time) control.

Important concepts, symbols, and equations

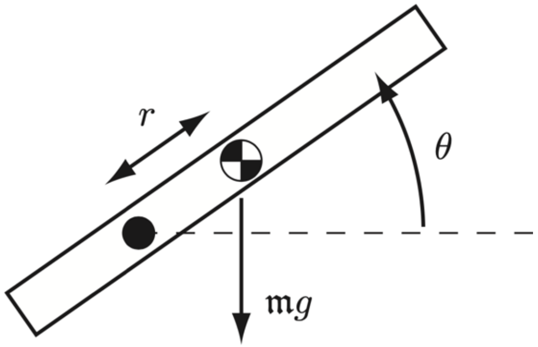
Example control objectives:

- **motion control**
- **force control**
- **hybrid motion-force control**
- **impedance control**

Types of control for the following tasks:

- Shaking hands with a human
- Erasing a whiteboard
- Spray painting
- Back massage
- Pushing an object across the floor with a mobile robot
- Opening a refrigerator door
- Inserting a peg in a hole
- Polishing with a polishing wheel
- Folding laundry

Important concepts, symbols, and equations



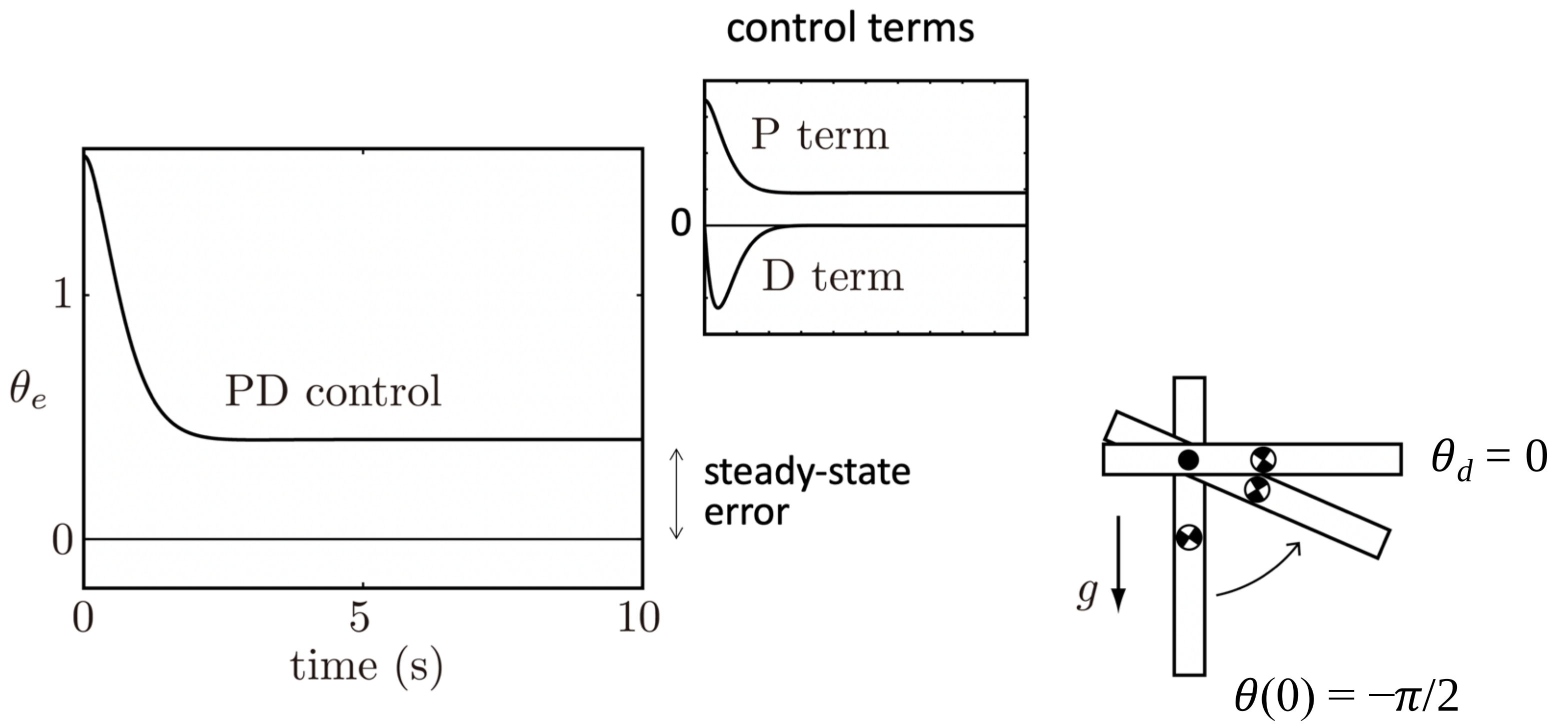
$$\tau = M\ddot{\theta} + \mathfrak{m}gr\cos\theta + b\dot{\theta}$$

$$\tau = M\ddot{\theta} + h(\theta, \dot{\theta})$$

Proportional-Integral-Derivative (PID) control

$$\tau = K_p\theta_e + K_i \int \theta_e(\mathrm{t})d\mathrm{t} + K_d\dot{\theta}_e$$

Important concepts, symbols, and equations (cont.)



Important concepts, symbols, and equations (cont.)

Setpoint PID control, $g \neq 0$

$$\tau = K_p\theta_e + K_i\int\theta_e(\mathrm{t})d\mathrm{t} + K_d\dot{\theta}_e$$

$$\tau = M\ddot{\theta} + \underbrace{\mathfrak{m}gr\cos\theta}_{\tau_{\mathrm{dist}}} + b\dot{\theta}$$

$$\dot{\theta}_d = \ddot{\theta}_d = 0$$

$$M\ddot{\theta}_e + (b+K_d)\dot{\theta}_e + K_p\theta_e + K_i\int\theta_e(\mathrm{t})d\mathrm{t} = \tau_{\mathrm{dist}}$$

$$M\theta_e^{(3)} + (b+K_d)\ddot{\theta}_e + K_p\dot{\theta}_e + K_i\theta_e = 0$$

$$s^3 + \frac{b+K_d}{M}s^2 + \frac{K_p}{M}s + \frac{K_i}{M} = 0$$

Important concepts, symbols, and equations (cont.)

What about tracking general trajectories, not just setpoint control?

$$\tau = M\Bigg(\underbrace{\ddot{\theta}_d + K_p\theta_e + K_i\int\theta_e \mathrm{d}t + K_d\dot{\theta}_e}_{\text{commanded } \ddot{\theta}}\Bigg) + h(\theta, \dot{\theta})$$

$$\ddot{\theta}_e = \ddot{\theta}_d - \ddot{\theta}$$

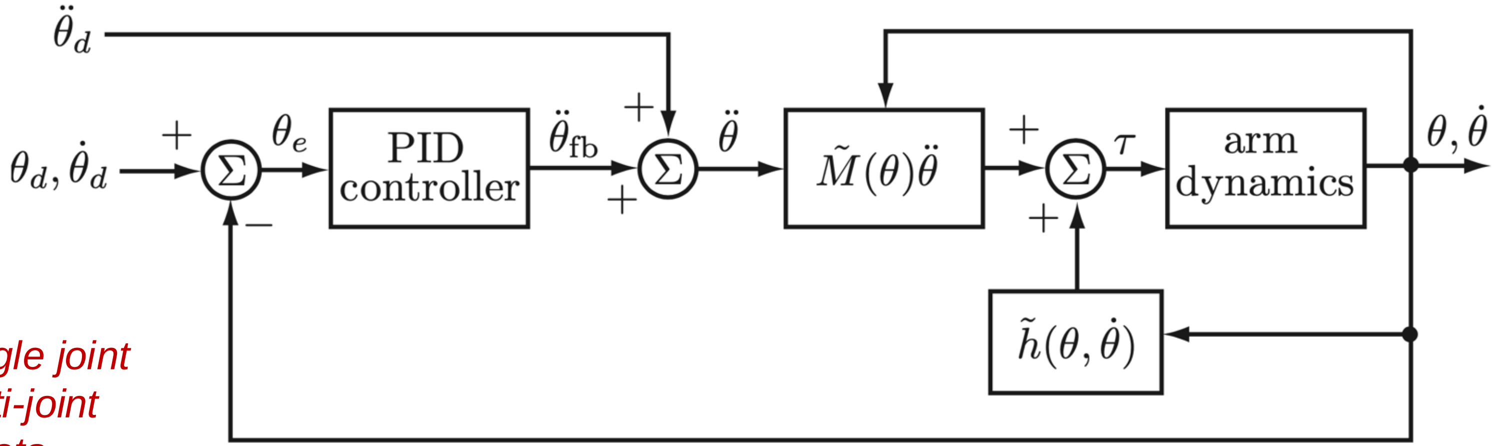
$$\ddot{\theta}_e = -K_d\dot{\theta}_e - K_p\theta_e - K_i\int\theta_e \mathrm{d}t$$

$$\theta_e^{(3)} + K_d\ddot{\theta}_e + K_p\dot{\theta}_e + K_i\theta_e = 0$$

<u>Important concepts, symbols, and equations</u> (cont.)

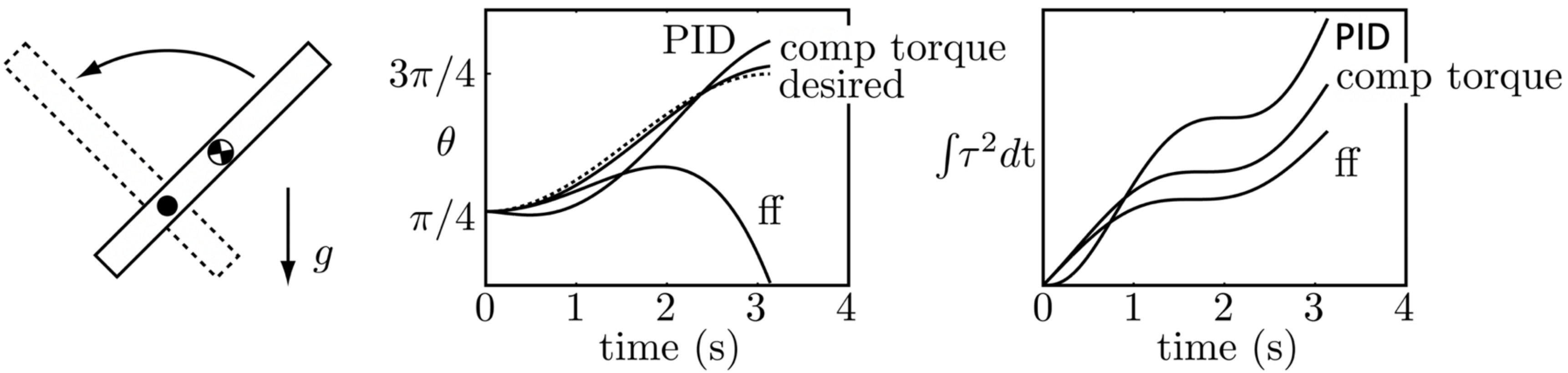
Computed torque control (feedback linearization)

$$\tau = \tilde{M}(\theta)\left(\ddot{\theta}_d + K_p\theta_e + K_i\int\theta_e(\mathrm{t})d\mathrm{t} + K_d\dot{\theta}_e\right) + \tilde{h}(\theta,\dot{\theta})$$



for a single joint or multi-joint robots

Important concepts, symbols, and equations (cont.)



Motion planning for robots

- Usually we assume the geometry of the environment is fully known.
- We want to find a path from the START position to the GOAL position while avoiding collision with obstacles.
- Solved by research in the 1980s and 1990s. Sampling based methods proved practical.

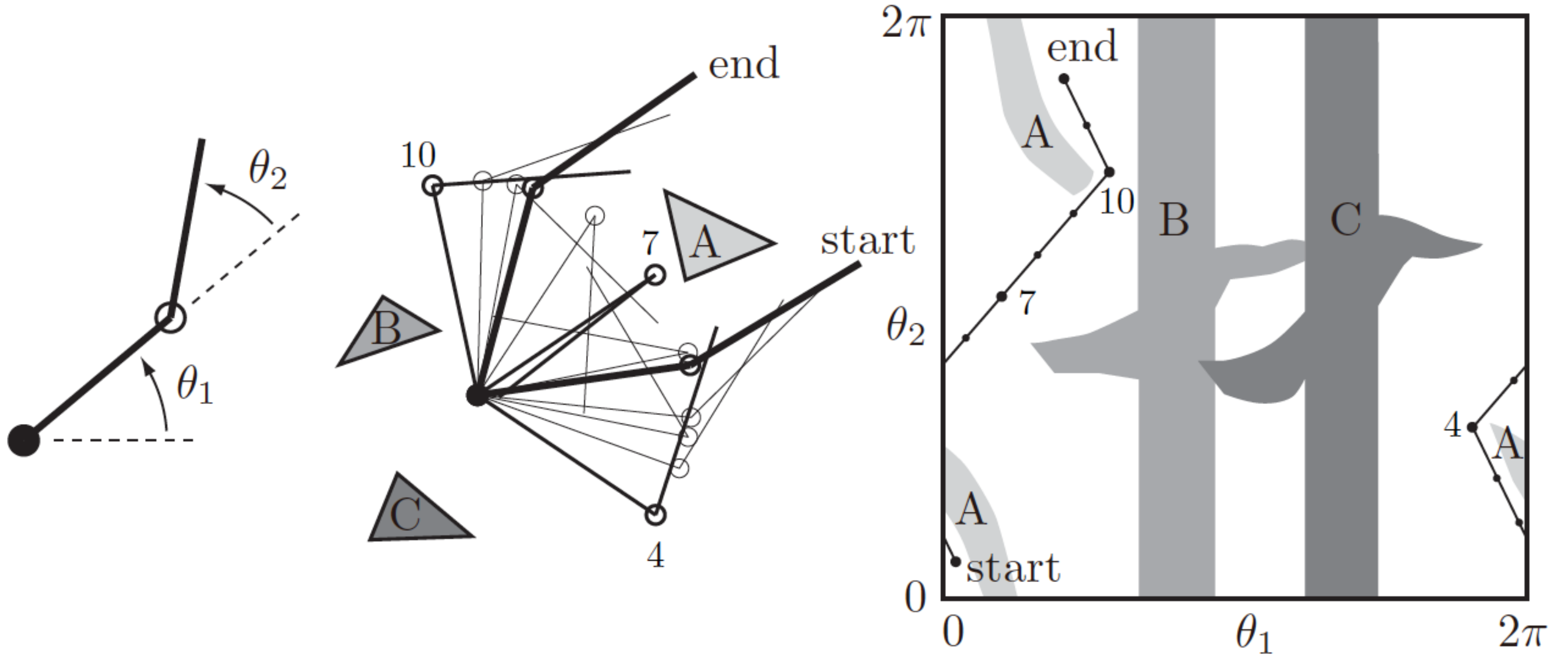


Figure 10.2: (Left) The joint angles of a 2R robot arm. (Middle) The arm navigating among obstacles A, B, and C. (Right) The same motion in C-space. Three intermediate points, 4, 7, and 10, along the path are labeled.

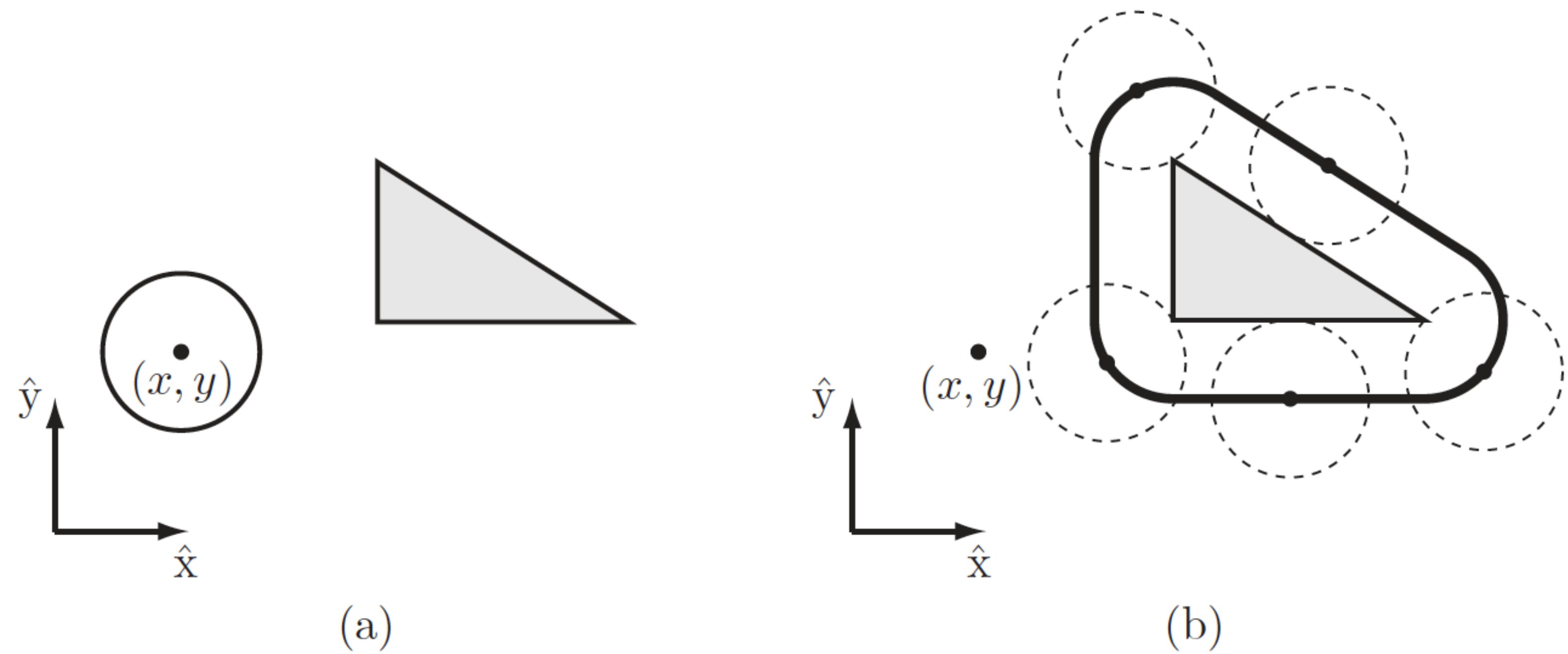


Figure 10.3: (a) A circular mobile robot (open circle) and a workspace obstacle (gray triangle). The configuration of the robot is represented by (x, y), the center of the robot. (b) In the C-space, the obstacle is "grown" by the radius of the robot and the robot is treated as a point. Any (x, y) configuration outside the bold line is collision-free.

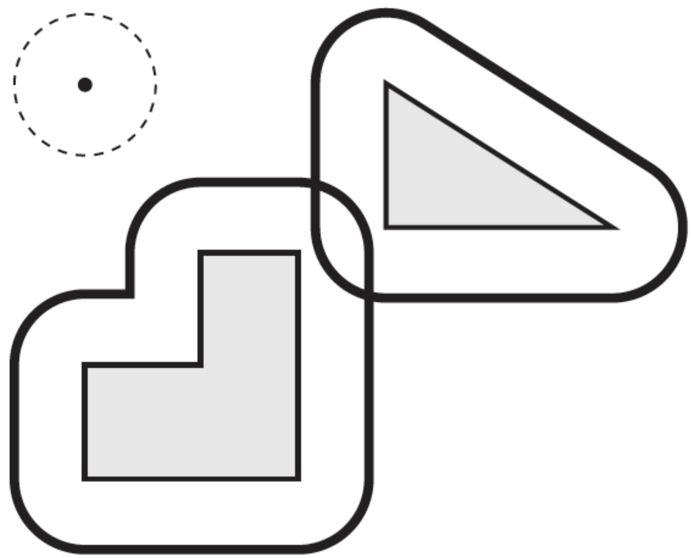


Figure 10.4: The "grown" C-space obstacles corresponding to two workspace obstacles and a circular mobile robot. The overlapping boundaries mean that the robot cannot move between the two obstacles.

Represent the free configuration C_{free} space by a 1D roadmap R

(a) **Reachability**. From every point $q \in \mathcal{C}_{\text{free}}$, a free path to a point $q' \in R$ can be found trivially (e.g., a straight-line path).

(b) **Connectivity**. For each connected component of $\mathcal{C}_{\text{free}}$, there is one connected component of R.

Separately find paths from START to R and GOAL to R, and link them via a path on R

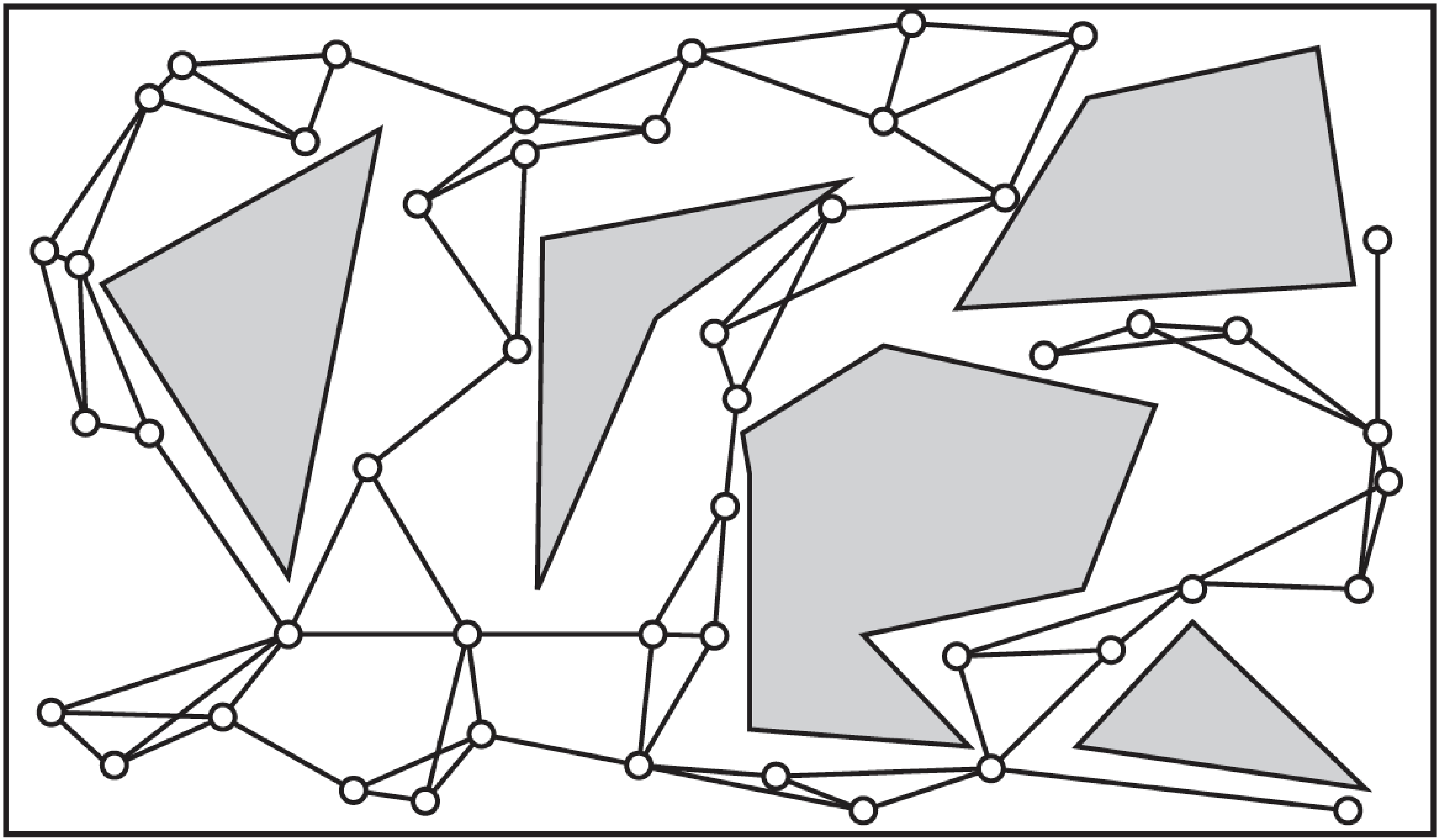


Figure 10.20: An example PRM roadmap for a point robot in $\mathcal{C} = \mathbb{R}^2$. The $k = 3$ closest neighbors are taken into consideration for connection to a sample node q. The

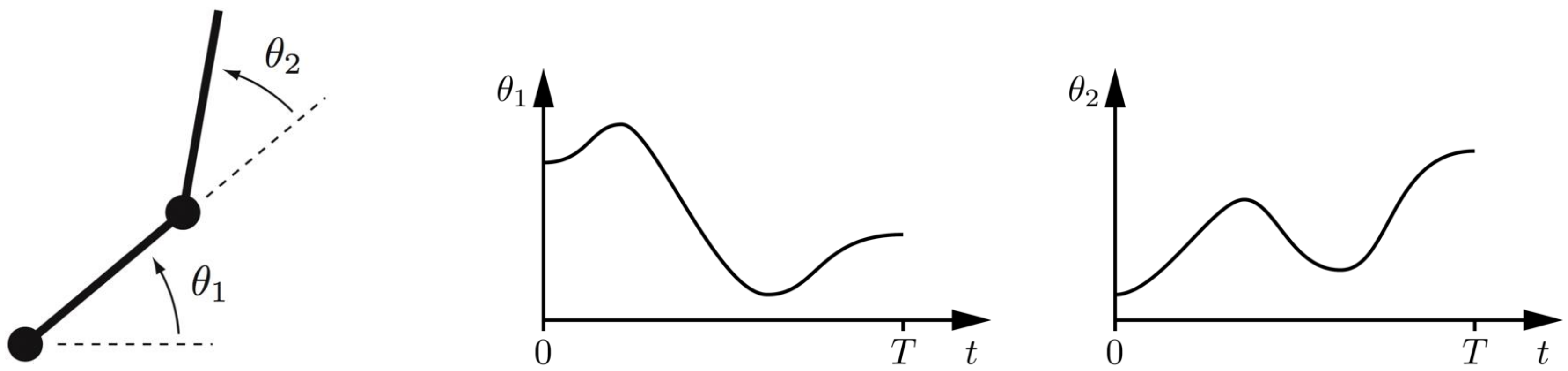
Algorithm 10.4 PRM roadmap construction algorithm (undirected graph).

for $i = 1, \ldots, N$ **do**
  $q_i \leftarrow$ sample from $\mathcal{C}_{\text{free}}$
  add q_i to R
end for
for $i = 1, \ldots, N$ **do**
  $\mathcal{N}(q_i) \leftarrow k$ closest neighbors of q_i
  for each $q \in \mathcal{N}(q_i)$ **do**
    if there is a collision-free local path from q to q_i and there is not already an edge from q to q_i **then**
      add an edge from q to q_i to the roadmap R
    end if
  end for
end for
return R

Next We need to go from paths to trajectories

Trajectory: A specification of the configuration as a function of time.

$$\theta(t),\ t \in [0, T]$$

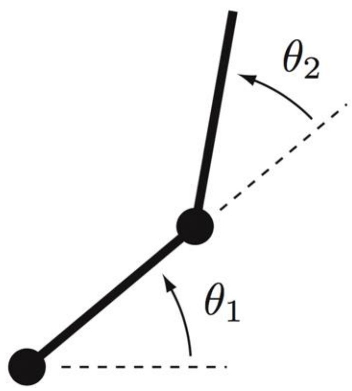


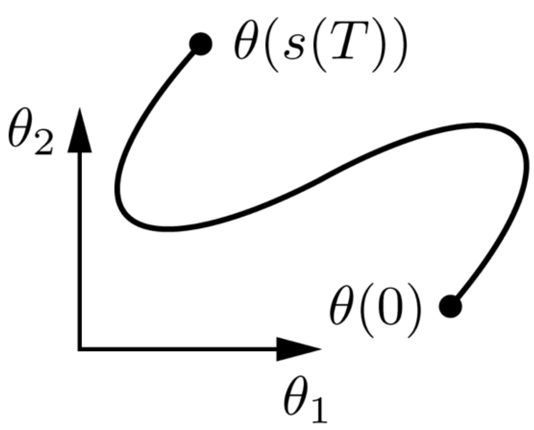
Important concepts, symbols, and equations (cont.)

Path: A specification of the configuration as a function of a path parameter.

$$\theta(s), \; s \in [0,1]$$

Time scaling: A mapping $s(t)$: $[0, T] \rightarrow [0, 1]$, from time to the path parameter.

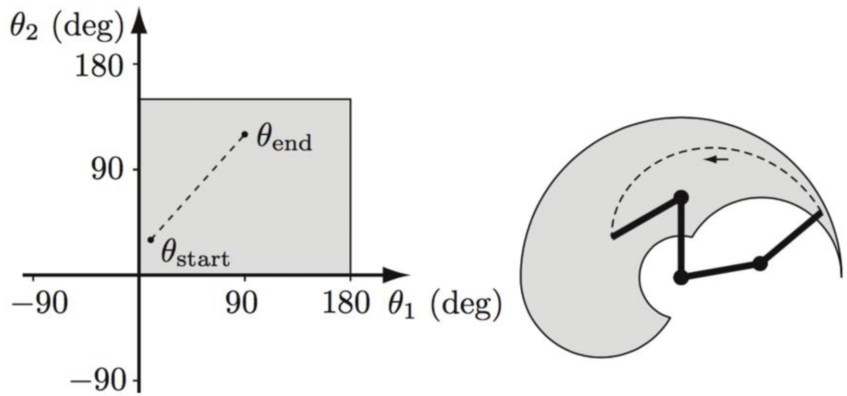




Important concepts, symbols, and equations (cont.)

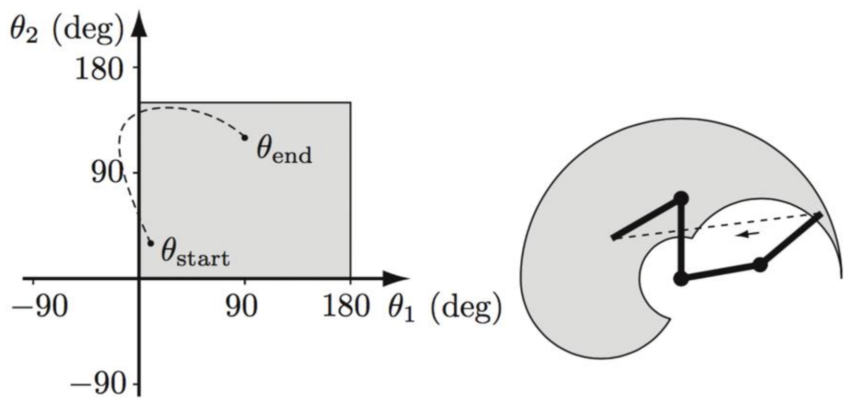
straight line in joint space

$$\theta(s) = \theta_{\text{start}} + s(\theta_{\text{end}} - \theta_{\text{start}})$$



straight line in task space

$$X(s) = X_{\text{start}} + s(X_{\text{end}} - X_{\text{start}})$$



An example of how to do the scaling in a smooth way.

S-curve time scaling